Esercitazione 2

Studio numerico di metodi a un passo

Sistemi Dinamici: Teoria e Metodi Numerici

Il problema

Si consideri il seguente sistema dinamico, dipendente dal parametro reale μ:

$$\begin{cases} \dfrac{dy_1}{dt} = \mu y_1 + y_2 - (y_1 + y_2)(y_1^2 + y_2^2), \\ \dfrac{dy_2}{dt} = -y_1 + \mu y_2 - (y_2 - y_1)(y_1^2 + y_2^2), \\ y_1(0) = y_1^0, \quad y_2(0) = y_2^0, \end{cases} \tag{1}$$

che scriviamo in forma compatta come $\mathbf{y}'(t) = \mathbf{f}(t, \mathbf{y}(t))$, $\mathbf{y}(0) = \mathbf{y}^0$, con $\mathbf{y} = (y_1, y_2)^T \in \mathbb{R}^2$.

Soluzione esatta. Per $\mu > 0$ il problema (1) ha la soluzione esatta

$$y_1(t) = r(t)\cos\theta(t), \qquad y_2(t) = r(t)\sin\theta(t),$$

dove

$$r(t) = \frac{\sqrt{\mu}\, r_0}{\sqrt{\varphi(t)}}, \qquad \theta(t) = \theta_0 - t + \mu\left(t + \frac{\ln\varphi(t) - \ln\mu}{2\mu}\right), \qquad \varphi(t) = r_0^2 + (\mu - r_0^2)\, e^{-2\mu t},$$

con $r_0 = \sqrt{(y_1^0)^2 + (y_2^0)^2}$ e θ_0 l'angolo polare del dato iniziale. Useremo questa soluzione per calcolare l'errore della soluzione numerica.

Osservazioni

- *Suggerimento per la verifica:* passando a coordinate polari $y_1 = r\cos\theta$, $y_2 = r\sin\theta$, il sistema (1) diventa
$$r' = \mu r - r^3, \qquad \theta' = r^2 - 1.$$
La prima è un'equazione di Bernoulli (si risolve con la sostituzione $z = r^{-2}$, oppure con la sostituzione $w = r^2$, che dà un'equazione logistica); la seconda si integra direttamente.
- In MATLAB si calcoli θ_0 con `atan2(y20,y10)` e non con `atan(y20/y10)`: quest'ultima restituisce l'angolo corretto solo se $y_1^0 > 0$.
- Per $\mu > 0$ si ha $r(t) \to \sqrt{\mu}$ per $t \to +\infty$: tutte le orbite (tranne l'equilibrio nell'origine) tendono al ciclo limite $r = \sqrt{\mu}$.

I metodi numerici

Si usa un passo costante h sull'intervallo $[0, T]$: i nodi sono $t_n = nh$, $n = 0, 1, \ldots, N$, con $N = T/h$, e $\mathbf{u}_n \approx \mathbf{y}(t_n)$.

a) **Eulero esplicito:**

$$\mathbf{u}_{n+1} = \mathbf{u}_n + h\,\mathbf{K}_1^n, \qquad \mathbf{K}_1^n = \mathbf{f}(t_n, \mathbf{u}_n).$$

b) **Runge–Kutta esplicito a due stadi**, con parametro $0 < \alpha \le 1$:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + h\left[\left(1 - \frac{1}{2\alpha}\right)\mathbf{K}_1^n + \frac{1}{2\alpha}\,\mathbf{K}_2^n\right], \qquad \begin{aligned} \mathbf{K}_1^n &= \mathbf{f}(t_n, \mathbf{u}_n), \\ \mathbf{K}_2^n &= \mathbf{f}(t_n + \alpha h,\ \mathbf{u}_n + \alpha h\,\mathbf{K}_1^n). \end{aligned}$$

Per $\alpha = 1$ si ottiene il metodo di Heun, per $\alpha = 1/2$ il metodo del punto medio (Eulero modificato).

Come si misurano gli errori

Si raccolgano la soluzione esatta e quella approssimata nei nodi in due matrici di dimensione $(N+1)\times 2$:

$$\texttt{ye} = \begin{pmatrix} y_1(t_0) & y_2(t_0) \\ \vdots & \vdots \\ y_1(t_N) & y_2(t_N) \end{pmatrix} \begin{matrix} \\ \leftarrow \text{istante } t_0 \\ \\ \leftarrow \text{istante } t_N \end{matrix} \qquad \texttt{ya} = \begin{pmatrix} u_{1,0} & u_{2,0} \\ \vdots & \vdots \\ u_{1,N} & u_{2,N} \end{pmatrix}.$$

Quindi **ogni riga corrisponde a un istante di tempo** e **ogni colonna a una componente**. (In MATLAB gli indici partono da 1: la riga $n+1$ contiene l'istante t_n.) Indichiamo con $\mathbf{e}_n = \mathbf{y}(t_n) - \mathbf{u}_n \in \mathbb{R}^2$ l'errore al tempo t_n, cioè la riga $n+1$ di $\texttt{ye} - \texttt{ya}$.

Si usano due tipi di norme, da non confondere:

1. **norma nelle componenti** (su ogni *riga*): per ogni istante t_n si calcola la norma del vettore $\mathbf{e}_n \in \mathbb{R}^2$. Il risultato è un *vettore* di $N+1$ valori, cioè una funzione del tempo, che si può disegnare in funzione di t;
2. **norma nel tempo** (su tutta la *colonna*): si prende un vettore di $N+1$ valori e se ne calcola la norma su tutti gli istanti. Il risultato è uno *scalare*, che riassume l'errore su tutto l'intervallo $[0, T]$.

Quantità	**Definizione**	**Dimensione**	**In MATLAB**
Errori nelle singole componenti			
$\texttt{er1}(n)$	$y_1(t_n) - u_{1,n}$	$N+1$	`er1 = ye(:,1) - ya(:,1)`
$\texttt{er2}(n)$	$y_2(t_n) - u_{2,n}$	$N+1$	`er2 = ye(:,2) - ya(:,2)`
Norme nelle componenti, istante per istante			
$E_\infty(n)$	$\|\mathbf{e}_n\|_\infty = \max\{\|\texttt{er1}(n)\|, \|\texttt{er2}(n)\|\}$	$N+1$	`Einf = max(abs(ye-ya),[],2)`
$E_2(n)$	$\|\mathbf{e}_n\|_2 = \sqrt{\texttt{er1}(n)^2 + \texttt{er2}(n)^2}$	$N+1$	`E2 = sqrt(sum((ye-ya).^2,2))`
$\texttt{nye}(n)$	$\|\mathbf{y}(t_n)\|_2 = r(t_n)$	$N+1$	`nye = sqrt(sum(ye.^2,2))`
$\texttt{nya}(n)$	$\|\mathbf{u}_n\|_2$	$N+1$	`nya = sqrt(sum(ya.^2,2))`
Norma nel tempo: un solo numero per ogni h			
$\texttt{err}(h)$	$\dfrac{\|E_2\|_2}{\|\texttt{nye}\|_2} = \left(\dfrac{\sum_{n=0}^{N} \|\mathbf{e}_n\|_2^2}{\sum_{n=0}^{N} \|\mathbf{y}(t_n)\|_2^2}\right)^{1/2}$	1	`err = norm(E2)/norm(nye)`

Attenzione. Nelle funzioni `max`, `sum` (e `vecnorm`, disponibile da MATLAB R2017b: `vecnorm(ye-ya,2,2)`, `vecnorm(ye-ya,Inf,2)`) l'argomento `2` indica la dimensione 2 (le colonne): l'operazione si fa *riga per riga*, cioè sulle componenti, e restituisce un valore per ogni istante. Invece `norm(ye-ya)` applicato a una matrice calcola la norma spettrale (il massimo valore singolare), che *non* è nessuna delle quantità richieste. Applicare `norm` solo a vettori, come `E2` e `nye`.

Osservazione. $\texttt{err}(h)$ è un errore *relativo*: sommando i quadrati di tutti gli elementi della matrice si ottiene

$$\texttt{err}(h) = \frac{\|\texttt{ye} - \texttt{ya}\|_F}{\|\texttt{ye}\|_F} \qquad \text{(in MATLAB: } \texttt{norm(ye-ya,'fro')/norm(ye,'fro')}\text{)},$$

dove $\|\cdot\|_F$ è la norma di Frobenius. Numeratore e denominatore crescono entrambi come N al diminuire di h, quindi il rapporto si può confrontare per passi diversi: approssima l'errore relativo $\|\mathbf{y} - \mathbf{u}\|_{L^2(0,T)} / \|\mathbf{y}\|_{L^2(0,T)}$.

Esercizi

1. **Soluzione esatta.** Verificare che le funzioni $y_1(t)$, $y_2(t)$ date sopra risolvono (1) (si veda il suggerimento). Scrivere una function MATLAB `ye = esatta(t,mu,y0)` che, dato il vettore colonna dei tempi `t`, restituisca la matrice `ye` di dimensione $(N+1)\times 2$.

2. **Implementazione.** Implementare in MATLAB i metodi a) e b) con passo costante h, ad esempio come function

   `[t,ya] = eulero(f,T,y0,h)`, `[t,ya] = rk2(f,T,y0,h,alpha)`,

   dove `f` è un function handle `f(t,y)` che restituisce un vettore colonna, `t` è il vettore colonna dei nodi e `ya` è una matrice $(N+1) \times 2$ con la stessa struttura di `ye`.

3. **Simulazione.** Calcolare la soluzione numerica per $\mu = 1$ sull'intervallo $[0, 20]$ con dato iniziale $y_1^0 = 1$, $y_2^0 = -3$ e passo $h = 0.01$, usando il metodo a) e il metodo b) con $\alpha = 1$ e con $\alpha = 1/2$. Ripetere con il metodo a) e $h = 0.1$, $h = 0.2$: che cosa si osserva? Perché?

4. **Errori.** Per ciascuna simulazione calcolare i vettori `er1`, `er2`, E_∞, E_2, `nye`, `nya` definiti nella tabella.

5. **Grafici.** Per ciascun metodo produrre con `subplot(2,2,k)` una figura con 4 pannelli:

   (i) le componenti in funzione di t: `ya(:,1)` e `ye(:,1)`, `ya(:,2)` e `ye(:,2)`;

   (ii) gli errori in funzione di t: `er1`, `er2`, E_∞, E_2;

   (iii) le norme in funzione di t: `nye` e `nya` (a che valore tendono?);

   (iv) le orbite nel piano delle fasi (y_1, y_2): quella approssimata (`ya(:,1)`, `ya(:,2)`) e quella esatta (`ye(:,1)`, `ye(:,2)`) (usare `axis equal`).

   Mettere in ogni pannello titolo, etichette degli assi e legenda.

6. **Ordine di convergenza del metodo a).** Calcolare `err`(h) per i passi $h_k = 0.02 \cdot 2^{-k}$, $k = 0, 1, \ldots, 7$. Riportare i risultati:

   - in un grafico in scala log-log (`loglog`) di `err`(h) in funzione di h, insieme a una retta di riferimento di pendenza 1 (ad esempio $C\,h$);
   - in una tabella con h_k, `err`(h_k) e l'ordine stimato

   $$p_k = \log_2 \frac{\texttt{err}(h_{k-1})}{\texttt{err}(h_k)}, \qquad k = 1, \ldots, 7.$$

7. **Confronto degli ordini.** Ripetere l'esercizio precedente per il metodo b) con $\alpha = 1$ e $\alpha = 1/2$. Riportare le tre curve nello stesso grafico log-log, con rette di riferimento di pendenza 1 e 2. Quali ordini si osservano?